



**MINISTERUL EDUCAȚIEI ȘI CERCETĂRII AL REPUBLICII MOLDOVA**

**Universitatea Tehnică a Moldovei**

**Facultatea Calculatoare, Informatică și Microelectronică**

**Departamentul Informatică și Ingineria Sistemelor**

## **RAPORT**

Lucrare de laborator nr.1

la cursul „Sisteme de operare”

A efectuat :

CR-231 f/r, **Litiuc Daniel**

A verificat:

**Kapusteanschi M**

**Chișinău 2026**



## Cuprins

|                          |   |
|--------------------------|---|
| Sarcina .....            | 3 |
| Codul sursa .....        | 3 |
| Rezultatul rulării ..... | 5 |
| Concluzie .....          | 5 |

## Sarcina

Creați o aplicație cu mai multe taimere utilizând clasele Timer și TimerTask în diferite moduri în grup:

1. Să reacționeze la un anumit interval de timp.
2. Să reacționeze la un anumit timp.
3. Să reacționeze cu o perioadă indicată.

## Codul sursa

```
import java.awt.FlowLayout;
import java.util.Date;
import java.util.Timer;
import java.util.TimerTask;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.SwingUtilities;

//TEst1

public class lab_1 {

    public static void main(String[] args) {
        SwingUtilities.invokeLater(new Runnable() {
            @Override
            public void run() {
                creeazaFereastră();
            }
        });
    }

    public static void creeazaFereastră() {
        JFrame fereastră = new JFrame("Laborator 1 - Timere");
        fereastră.setLayout(new FlowLayout());
        fereastră.setSize(550, 150);
        fereastră.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JButton b1 = new JButton("Dupa 5 secunde: 0");
        JButton b2 = new JButton("La ora stabilita: 0");
        JButton b3 = new JButton("Periodic: 0");

        fereastră.add(b1);
        fereastră.add(b2);
        fereastră.add(b3);

        fereastră.setLocationRelativeTo(null);
        fereastră.setVisible(true);
    }
}
```

```

Timer t1 = new Timer();
Timer t2 = new Timer();
Timer t3 = new Timer();

// Prima actiune: dupa 5 secunde.
t1.schedule(new TimerTask() {
    @Override
    public void run() {
        SwingUtilities.invokeLater(new Runnable() {
            @Override
            public void run() {
                b1.setText("Dupa 5 secunde: 1");
            }
        });

        t1.cancel();
    }
}, 5000);

// A doua actiune: la un moment stabilit.
Date ora = new Date(System.currentTimeMillis() + 10000);
b2.setToolTipText("Ora programata: " + ora);

t2.schedule(new TimerTask() {
    @Override
    public void run() {
        SwingUtilities.invokeLater(new Runnable() {
            @Override
            public void run() {
                b2.setText("La ora stabilita: 1");
            }
        });

        t2.cancel();
    }
}, ora);

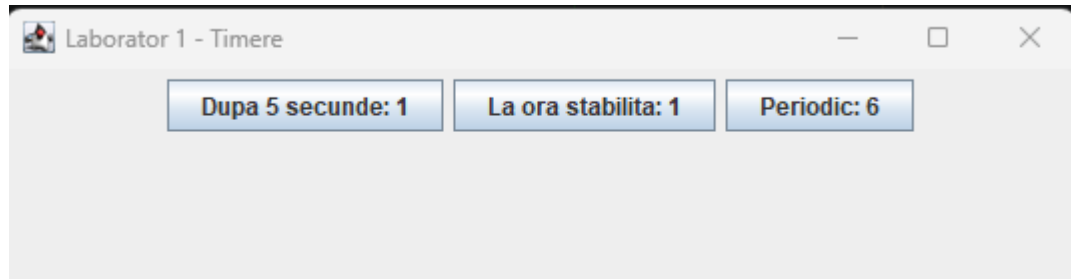
// A treia actiune: la fiecare 2 secunde.
t3.scheduleAtFixedRate(new TimerTask() {
    private int contor = 0;

    @Override
    public void run() {
        SwingUtilities.invokeLater(new Runnable() {
            @Override
            public void run() {
                contor++;
                b3.setText("Periodic: " + contor);
            }
        });
    }
});

```

```
    }  
    }, 2000, 2000);  
}  
}
```

## Rezultatul rulării



## Concluzie

În cadrul lucrării de laborator am realizat o aplicație Java cu interfață grafică Swing, utilizând clasele `Timer` și `TimerTask` pentru planificarea acțiunilor. Am implementat trei moduri de execuție: după o întârziere, la un moment calendaristic stabilit și periodic. Rezultatele au fost afișate prin actualizarea valorilor unor butoane, folosind `SwingUtilities.invokeLater()` pentru efectuarea modificărilor pe firul interfeței grafice. De asemenea, am exersat utilizarea Git și lucrul cu o ramură separată pentru gestionarea versiunilor proiectului. Lucrarea mi-a permis să înțeleg diferența dintre definirea unei acțiuni și planificarea executării acesteia, precum și importanța gestionării corecte a timerelor.